

# Numerical Stability and Stiffness: An Analysis of Eulers Method in Linear Differential Equations

Lucas Selvik  
McKelvey School of Engineering  
Washington University in St. Louis  
St. Louis, MO  
selvik@wustl.edu

Rex Paster   
McKelvey School of Engineering  
Washington University in St. Louis  
St. Louis, MO  
p.rex@wustl.edu

## I. INTRODUCTION

The goal of this case study is to demonstrate both understanding and ability in the tools explored in the Differential Equations and Dynamical Systems Modeling in Engineering course (ESE 2170). By critically evaluating two families of differential equations, this study demonstrates such knowledge, exploring solver algorithmic limitations of Euler's method, differential behaviors, and real world applications.

## II. EXPONENTIAL GROWTH & DECAY FUNCTION

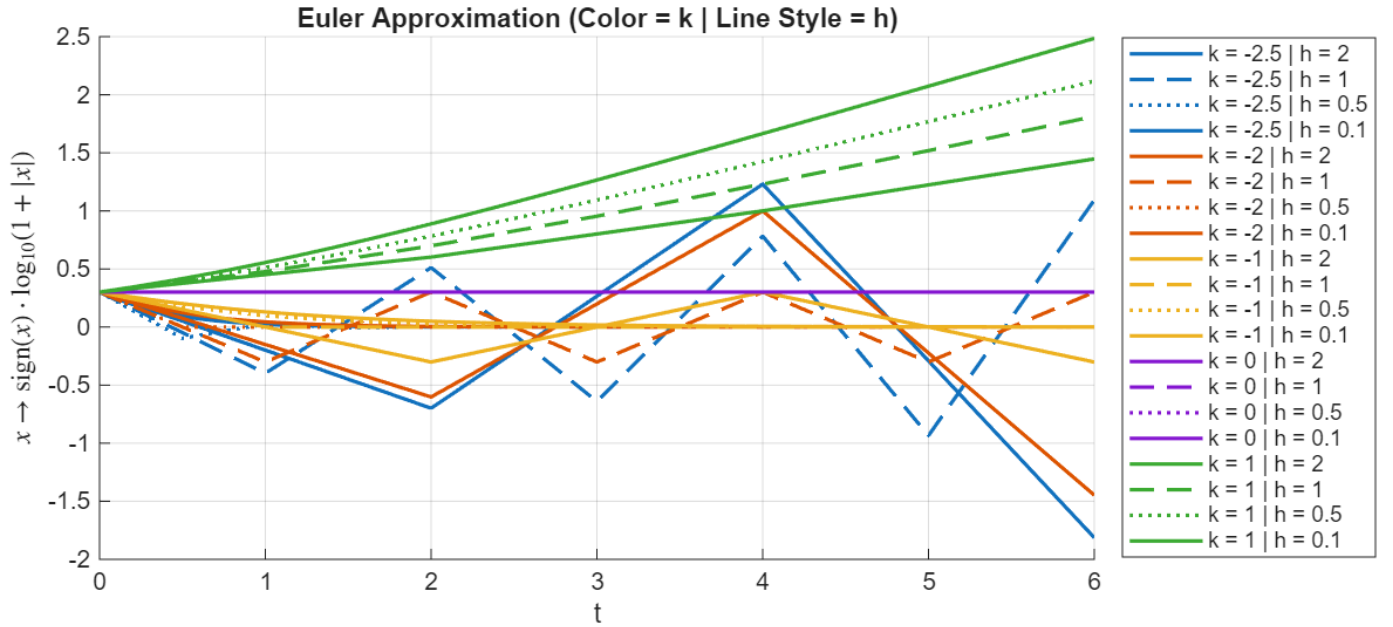


Fig. 1. A logarithmically scaled graph of the behavior of Euler's evaluation method when applied to the Exponential Growth & Decay Function with different step sizes and constants ( $k$ ).

The first family of equations that this study sets out to evaluate is the Exponential Growth/Decay family of functions defined by

$$x' = kx \quad \text{where } k \in \mathbb{R} \quad (1)$$

The first step of analyzing this family of equations was to find a solution to the family, providing a baseline expected behavior to be compared to solver behaviors. The full derivation, included in Appendix A, yields the following solution:

$$x(t) = x_0 e^{kt} \quad (2)$$

With the aforementioned derived solution, the next step was to explore behavior for different values of  $k$ .

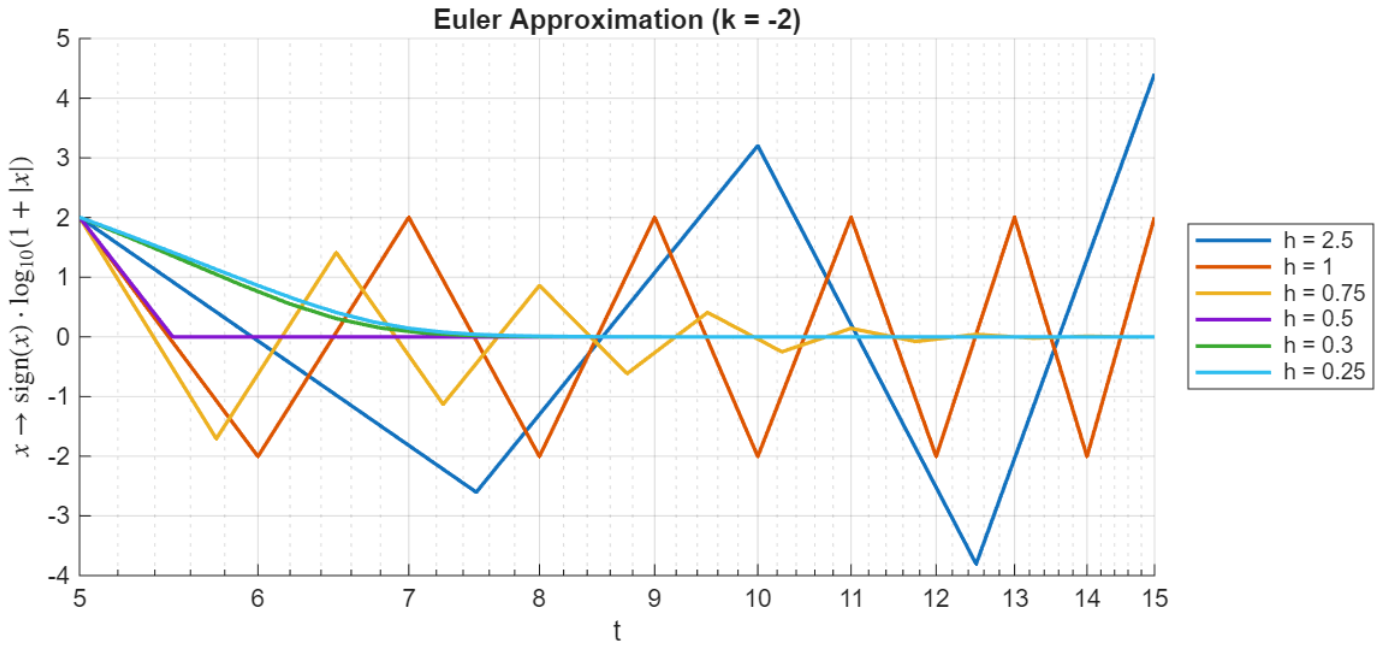


Fig. 2. A logarithmically scaled graph of the behavior of Euler's evaluation method when applied to the Exponential Growth & Decay Function with different step sizes for  $k = -2$ .

#### A. $k = -2$ (i.e. $k < 0$ )

In this subsection, we will evaluate the differential equation predominantly using the example of  $k = -2$ ; however, this section applies for any negative value of  $k$ .

When plugging  $k = -2$  into (1), we see that the differential equation has a stable equilibrium along the line  $x = 0$ . Any initial value  $x \neq 0$  will exhibit asymptotic behavior approaching  $x = 0$  as  $t \rightarrow \infty$ . If the initial value is equal to 0, then for all  $t$ ,  $x$  remains equal to zero.

Upon examining Figure 2, we observe that the expected behavior is not always accurately reproduced by Euler's method. Instead, the qualitative behavior of the numerical solution depends on the step size  $h$ .

For  $k = -2$  (or any negative value of  $k$ ), the correct behavior is represented when

$$h \in \left(0, -\frac{1}{k}\right] = (0, 0.5]$$

When  $h > -\frac{1}{k}$ , the method exhibits qualitatively different behaviors depending on the value of  $h$ :

- If  $h \in \left(-\frac{1}{k}, -\frac{2}{k}\right)$ , the numerical solution oscillates with a damping effect.
- If  $h = -\frac{2}{k}$ , the solution oscillates indefinitely between positive and negative values with constant magnitude.
- If  $h > -\frac{2}{k}$ , the solution oscillates with growing amplitude, leading to divergence (blow-up).

When modeling real-world systems, we typically do not have control over the value of  $k$ . Given the stability restrictions derived above, this presents a significant limitation of Euler's method.

If  $k$  is large and negative, the range of step sizes that prevent oscillatory behavior is

$$h \in \left(0, -\frac{1}{k}\right)$$

Notably, when  $|k|$  is large, this interval becomes extremely small, forcing  $h$  to be very small in order to maintain stability. Consequently, the method requires many more time steps, increasing computational cost and amplifying the accumulation of rounding errors.

Even if oscillations are acceptable and we only require the solution to eventually decay to zero, the admissible range expands only to

$$h \in \left(0, -\frac{2}{k}\right)$$

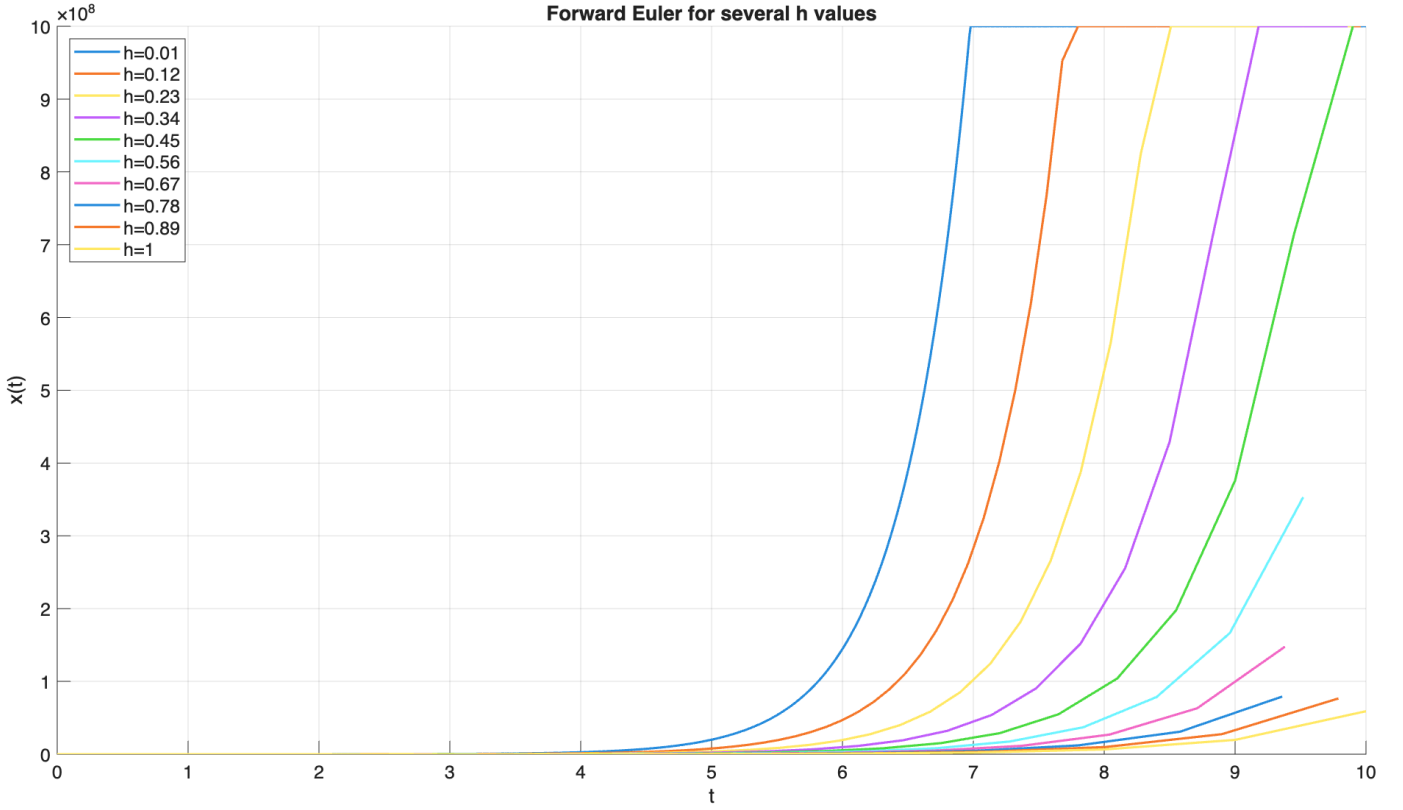


Fig. 3. Forward Euler model of  $x' = 2x$  for various  $h$  values.

However, for large negative  $k$ , this interval remains very small. Thus, the method still demands a small step size, resulting in substantial computational intensity despite the relaxed stability requirement. With this in mind, in addition to the knowledge that  $k$  is determined by the system we are modeling, the shortcomings of the Euler method become apparent.

Additionally, this family of differential equations has a unique solution for a given initial condition (as shown in Appendix A).

Since the equation has a unique solution, we know that the system itself behaves smoothly and predictably. Therefore, any instability, oscillation, or divergence observed in the Euler approximations arises from the numerical solving method rather than from the behavior of the differential equation itself.

#### B. $k = 2$ (i.e. $k > 0$ )

In this subsection, we will evaluate the differential equation predominantly using the example of  $k = 2$ ; however, this section applies for any positive value of  $k$ .

Following the calculations in the Appendix, integrating both sides of the equation  $x' = 2x$  will reveal solutions that take the form

$$x(t) = x_0 e^{2t}$$

As  $t \rightarrow \infty$ , we see that  $x(t) \rightarrow x_0 \cdot \infty$ . Therefore, if  $x_0 > 0$ , then  $x(t) \rightarrow +\infty$ , and if  $x_0 < 0$ , then  $x(t) \rightarrow -\infty$ . In either case, the solution is monotonic since there are no sign changes. If  $x_0 = 0$ ,  $x(t) = 0$  for  $t \in \mathbb{R}$ .

Euler's Method gives

$$x_{n+1} = x_n + hf(t_n, x_n)$$

Since  $f(t, x) = 2x$ , this becomes

$$x_{n+1} = x_n + 2hx_n = (1 + 2h)x_n$$

After  $n$  steps,

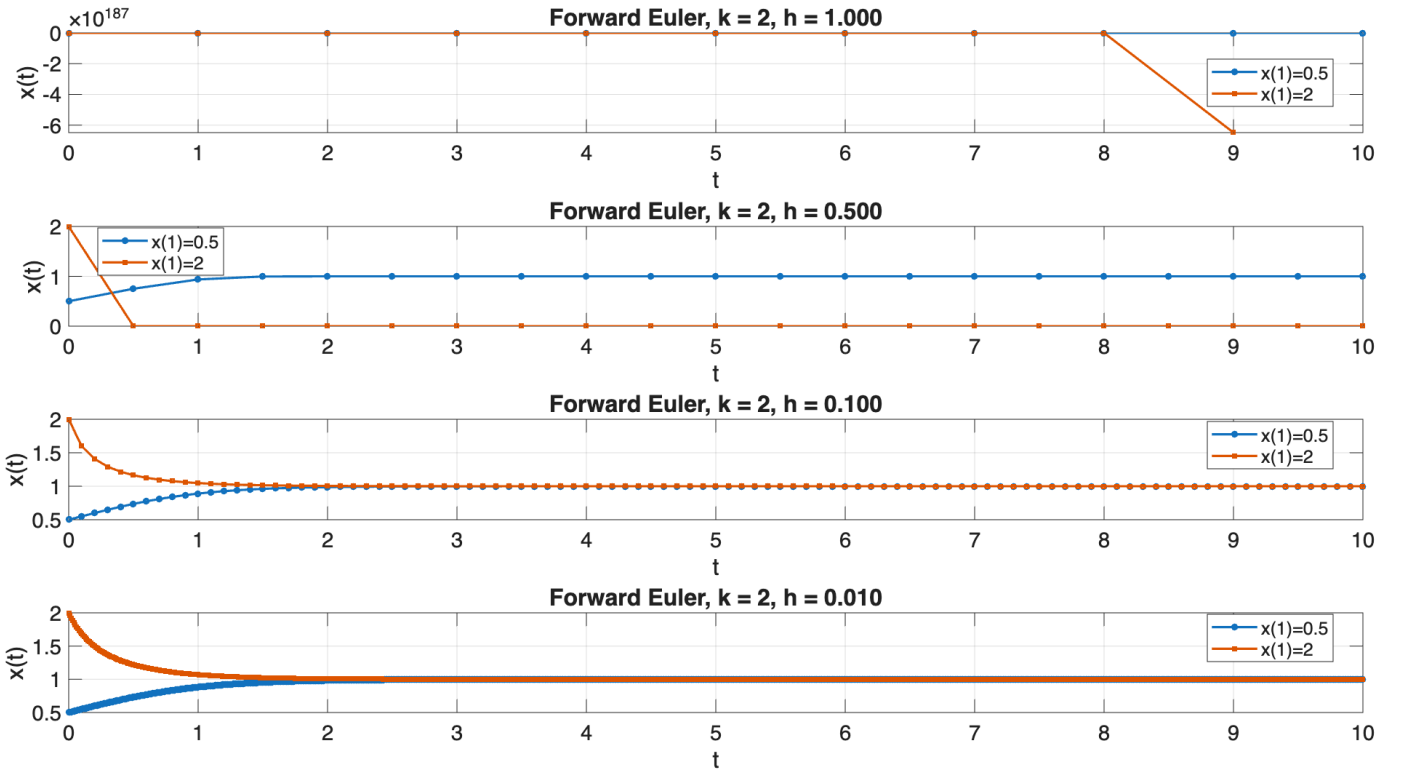


Fig. 4. Forward Euler model of  $x' = 2x(1-x)$  for various  $h$  values. Two solutions are modeled for each  $h$  value:  $0 < x_0 < 1$  and  $x_0 > 1$ .

$$x_n = (1 + 2h)^n x_0$$

We want the numerical solution to diverge to infinity monotonically. Therefore, we require  $1 + 2h > 1$ . If  $-1 \leq 1 + 2h \leq 1$ , the solution will not diverge to infinity. If  $1 + 2h < 0$ , the solution will alternate signs at each step. Solving for  $h$ , we conclude that in order for Euler's method to reflect the behavior of the true solution, we must have  $h > 0$ .

This behavior is consistent with an unrestricted population model; that is, the death rate is negligible and the birth rate is proportional to the current population.

When  $h < 0$ , we are no longer utilizing the Forward Euler Method. Instead, we are using a Backward Euler Method; however, this will break our system since time moves forward. If  $h = 0$ , time will not take discrete steps and will remain stuck. Conveniently, real world models will show the correct long-term behavior for all  $h > 0$ . Larger  $h$  values will have considerably more error but will follow the correct relationship; this is reflected in Figure 3.

### III. LOGISTIC FUNCTIONS

We can perform similar analysis on different families of differential equations; one such family is  $x' = kx(1-x)$ .

#### A. Differential Equation Family

We consider the Logistic Growth model, a non-linear family of differential equations defined by

$$\frac{dx}{dt} = kx(1-x) \quad (3)$$

where  $k$  is the growth rate parameter and  $x(t)$  represents the population as a fraction of the carrying capacity.

#### B. Stability and Step Size Analysis

If  $k > 0$ , solutions to the differential equation are found by solving  $0 = x(1-x)$ , producing  $x = 0$  and  $x = 1$ . Any  $0 < x_0 < 1$  and  $x_0 > 1$  will converge toward  $x = 1$ , signifying that  $x = 1$  is stable. On the other hand, any  $x_0 < 0$  will diverge from  $x = 0$ , signifying that  $x = 0$  is unstable.

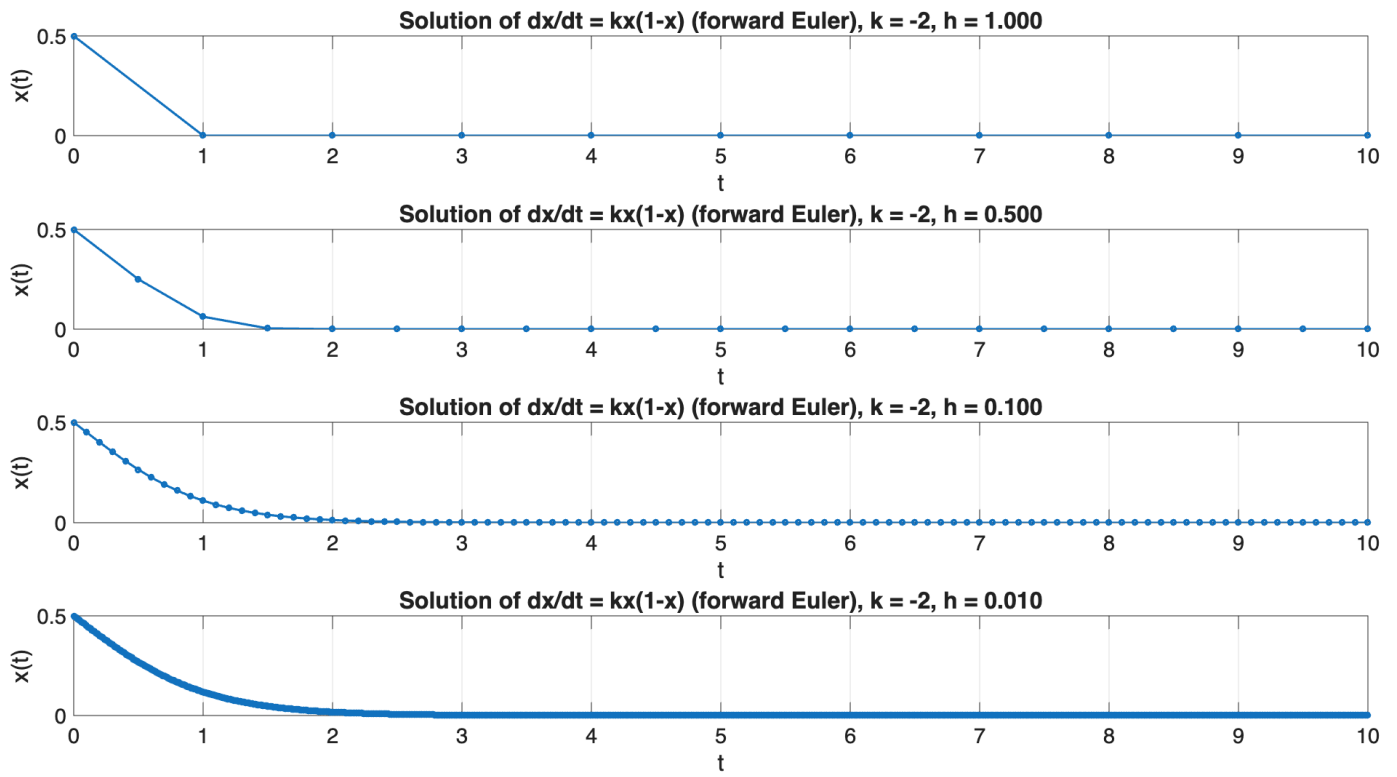


Fig. 5. Forward Euler model of  $x' = -2x(1 - x)$  for various  $h$  values and  $0 < x_0 < 1$ .

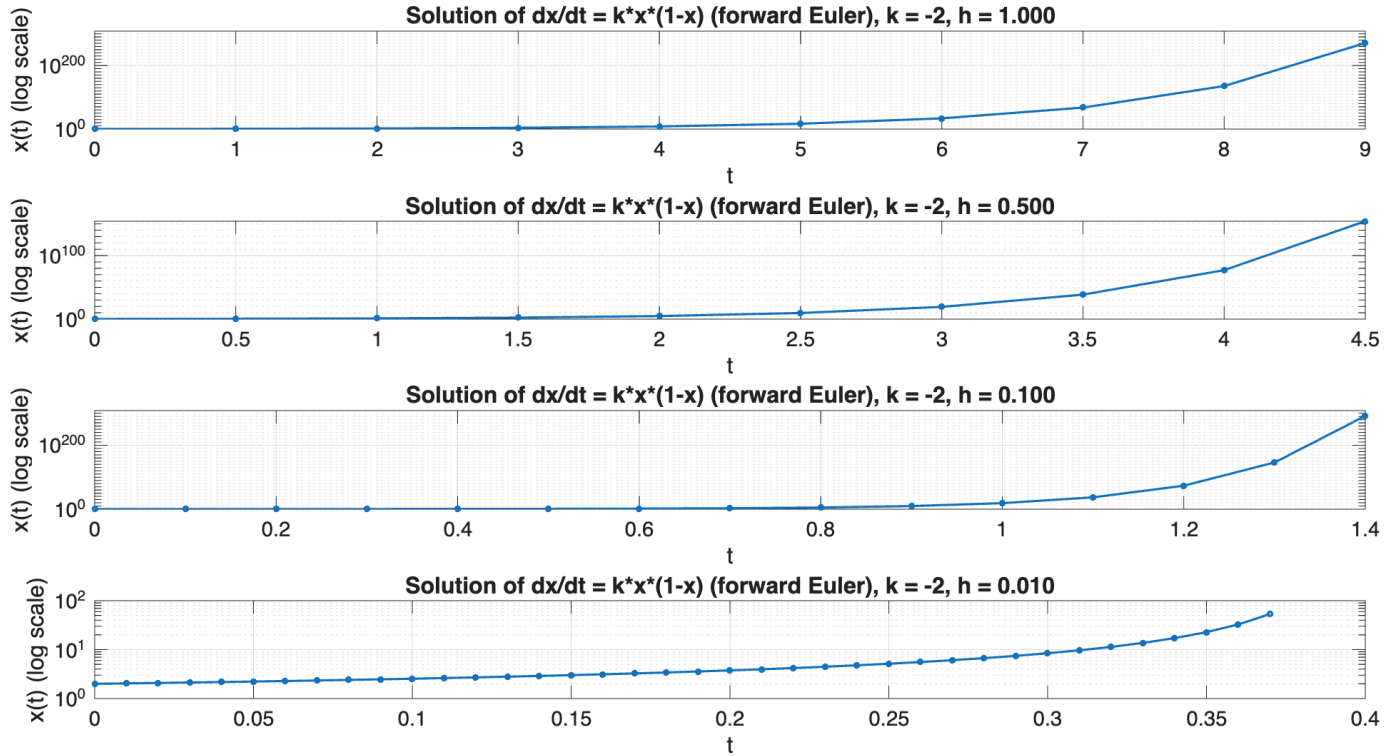


Fig. 6. Forward Euler model of  $x' = -2x(1 - x)$  for various  $h$  values and  $x_0 > 1$ .

If  $k < 0$ , solutions are found from  $0 = x(x - 1)$ , also giving  $x = 0$  and  $x = 1$ . However, the stability is reversed in this case. Any  $0 < x_0 < 1$  and  $x_0 < 0$  will converge toward  $x = 0$ , signifying that  $x = 0$  is stable. Any  $x_0 > 1$  will diverge from  $x = 1$ , signifying that  $x = 1$  is unstable.

To determine the range of  $h$  for which the model captures the physical behavior near the stable equilibrium  $x = 1$  (for  $k > 0$ ), we linearize the error. Let  $x_n = 1 + \epsilon_n$ , where  $\epsilon_n$  is a small numerical error.

Substituting into the Euler update,

$$\begin{aligned} 1 + \epsilon_{n+1} &= (1 + \epsilon_n) + hk(1 + \epsilon_n)(1 - (1 + \epsilon_n)) \\ 1 + \epsilon_{n+1} &= 1 + \epsilon_n - hk\epsilon_n(1 + \epsilon_n) \end{aligned}$$

Because  $\epsilon_n$  is a minuscule error, we can disregard  $O(\epsilon_n^2)$  terms:

$$\epsilon_{n+1} \approx \epsilon_n(1 - hk) \quad (4)$$

For stability, we require  $|\epsilon_{n+1}| \leq |\epsilon_n|$  (the error does not compound), which implies  $|1 - hk| \leq 1$ . For  $k > 0$ , this yields

$$0 < h < \frac{2}{k} \quad (5)$$

Similarly, for  $k < 0$ , the stable equilibrium is  $x = 0$ . Linearizing near  $x = 0$  gives

$$x_{n+1} \approx x_n(1 + hk) \implies 0 < h < -\frac{2}{k} \quad (6)$$

### C. Analysis of Numerical Results

The results from varying step sizes  $h$  demonstrate the relationship between the stability bound  $h < 2/|k|$  and the qualitative accuracy of the Forward Euler method. In Figure 4, using parameters  $k = 2$  and  $h = 1$  exhibits a numerical blow-up followed by a crash, confirming that the solver cannot function when the step size is outside the limit. Interestingly,  $h = 0.5$  shows the solution with  $x_0 = 2$  overshoot the target and land directly on the unstable equilibrium  $x = 0$ , illustrating a kind of trap where the solver is stable but physically inaccurate. As  $h$  decreases to 0.1 and 0.01, the expected asymptotic approach to the carrying capacity  $x = 1$  is correctly resolved.

For the scenario where  $k = -2$  shown in Figure 5, the results confirm that smaller step sizes are required to capture the smooth exponential decay toward the stable equilibrium at  $x = 0$ . Finally, Figure 6 illustrates the divergence of the system when  $k < 0$  and  $x_0 > 1$ . The logarithmic plots show that while the solver correctly identifies this explosion toward infinity, smaller values of  $h$  track the rapid growth with higher precision, allowing the solution to diverge earlier in simulation time. Collectively, these figures highlight the challenges of modeling stiff systems, where the necessity for stability and local accuracy forces a significantly restricted choice of  $h$ .

### D. Discussion of Stiffness

In the real world, this model represents populations with limited resources. Stiffness arises when  $k$  is large. In such cases, there should be a rapid approach to equilibrium, but the numerical solver requires an extremely small  $h$  to avoid oscillations or divergence. Failing to meet the  $h < 2/|k|$  criteria can lead to false extinctions or chaotic values.

## IV. NOTES ON THE UTILIZATION OF AI

Artificial intelligence tools, including ChatGPT, were utilized during the planning and development of this case study. These tools assisted with grammar refinement, formatting, reorganization, and logical consistency checks. All generated content was thoroughly reviewed, edited, and verified prior to inclusion. This paper represents the result of substantial intellectual contribution by Lucas Selvik and Rex Paster, and all submitted materials have been carefully reviewed to ensure accuracy and originality.

The following prompts were used during development:

- “give me the derivation of the exponential growth and decay functions solution in  $\LaTeX$ ” (The original derivation was completed on paper, and the  $\LaTeX$  version produced by generative AI was used as a way to easily get the equations used in the derivation into  $\LaTeX$ )
- “rewrite this for better flow and in  $\LaTeX$ ”
- “You are a Professor Grading this paper, your feedback is a copy of the paper with changes you would make implemented. I’m going to provide you a text and I want you to implement changes to help flow without adjusting the content or tone. If you see a spot that cannot work without extra info, mark it in bold, don’t add or remove any work.”
- “without changing any content, make the organization consistent”
- “translate to overleaf code”

## APPENDIX A

### DERIVATION OF A SOLUTION FOR THE EXPONENTIAL GROWTH & DECAY FUNCTION

#### *Existence and Uniqueness*

For  $f(x, t) = kx$ , both  $f$  and  $\frac{\partial f}{\partial x}$  are continuous for all  $(x, t) \in \mathbb{R}^2$ . Therefore, the conditions of the Existence and Uniqueness Theorem are satisfied, and the initial value problem

$$x' = kx, \quad x(0) = x_0$$

has a unique solution.

#### *Finding the Solution*

Given the following differential equation

$$x' = kx$$

where

- $x(t)$  is the value at time  $t$
- $k$  is a constant

#### *Step 1: Separate Variables*

$$\frac{dx}{dt} = kx$$

$$\frac{dx}{x} = k dt$$

#### *Step 2: Integrate Both Sides*

$$\int \frac{1}{x} dx = \int k dt$$

$$\ln |x| = kt + C$$

where  $C$  is the constant of integration.

#### *Step 3: Solve for $x$*

Exponentiate both sides:

$$|x| = e^{kt+C}$$

Using the property  $e^{a+b} = e^a e^b$ :

$$|x| = e^C e^{kt}$$

Let  $A = e^C$  (a positive constant). Removing the absolute value by absorbing the sign into  $A$ :

$$x(t) = Ae^{kt}$$

#### *Step 4: Apply Initial Condition*

If  $x(0) = x_0$ , then

$$x_0 = Ae^{k(0)} = A$$

So the solution becomes

$$\boxed{x(t) = x_0 e^{kt}}$$

---

# Table of Contents

|                           |   |
|---------------------------|---|
| Part 1 .....              | 1 |
| CHATGPT UTILIZATION ..... | 3 |
| Part 2 .....              | 3 |
| Part 3 .....              | 3 |
| Part 4 .....              | 5 |
| Part 5: .....             | 5 |

## Part 1

### Euler's Method

```
figure();

% Settings
x_prime = @(t, x, k) k.*x; %1st Order Expression to solve
x_0 = 1; % initial x
k_list = [-2.5, -2, -1, 0, 1]; %values for k
h_list = [2, 1, 0.5, 0.1]; % step sizes
t_final = 6; %Upper Bound of Evaluation

legend_list = strings(1, numel(k_list)*numel(h_list)); %Initialize the
Legend Naming System
legend_index = 1;

%Visual Settings
colors = lines(length(k_list)); %Color for each k value
line_styles = ["-", "--", ":", "-"]; %Line style for each h value

hold on; %Plot All on Same Graph

for k_index = 1:length(k_list) %Iterate though each k value

    k = k_list(k_index);
    color = colors(k_index,:); %Assign color based on k

    for h_index = 1:length(h_list) %Iterate though each h value

        h = h_list(h_index);
        style = line_styles(h_index); %Assign line style based on h

        %Create time range
        t = 0:h:t_final;

        %Prepare Internal Vars
        x = zeros(size(t));
        n = numel(t);

        x(1) = x_0; % set x(1) to initial x
```



---

```

    %Run Euler Iteration
    for i=1:n-1
        x(i+1) = x(i) + h * x_prime(t(i), x(i), k); %Evaluate the DE
using Euler's Method
    end

    %Apply symmetric log transform for +/- values
    x_plot = sign(x) .* log10(1 + abs(x)); % symlog style

    %Plot Approximations
    plot(t, x_plot, 'Color',color, 'LineStyle',style, 'LineWidth',1.5)

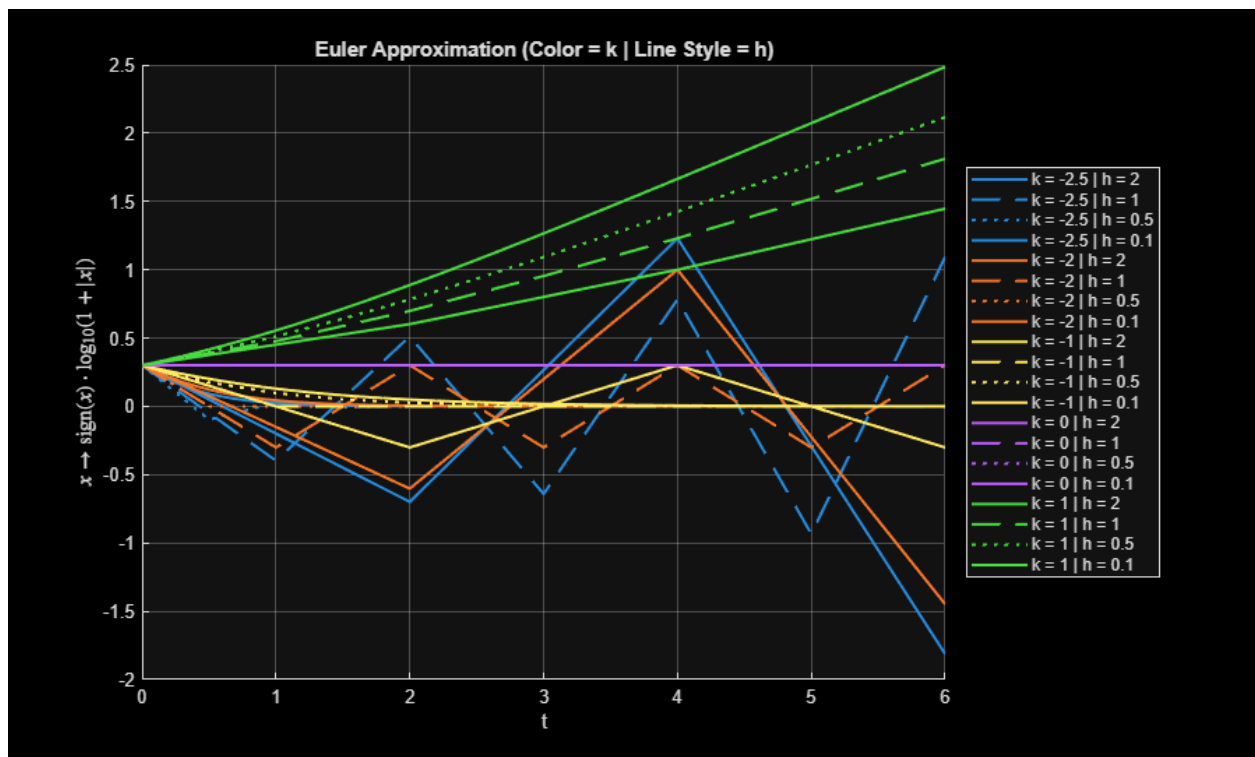
    %Append Legend Name
    legend_list(legend_index) = ("k = " + k + " | h = " + h);
    legend_index = legend_index + 1;

end
end

hold off;

%Graph Settings
ylabel('$x \rightarrow \mathrm{sign}(x) \cdot \log_{10}(1 + |x|)$', 'Interpreter', 'latex'); %LaTeX y-axis
xlabel('t'); %x-axis label
title('Euler Approximation (Color = k | Line Style = h)') %Plot title
legend(legend_list, 'Location', 'eastoutside') %Add legend
grid on %Turn on grid

```



---

# CHATGPT UTILIZATION

"can i make this axis exponential" "how can i make this exponential in both positive and negative axis" "help make this graph more digestable"

## Part 2

Q: When  $k = -2$  what do you expect to happen to a solution  $x(t)$  as  $t$  approaches infinity? Describe some real-world situations where you expect to see this type of behaviour.

```
% R: When k = -2, the DE will have a stable equilibrium for all values
%   on the line x = 0. Any initial value x != 0 will have an asymptotic
%   behavior approaching x = 0 as t --> infinity. If the initial value is
%   equal to 0, for all t x will remain equal to zero as t --> infinity.
```

## Part 3

Euler's Method

```
figure();

% Settings
x_prime = @(t, x, k) k.*x; %1st Order Expression to solve
x_0 = 100; % initial x
k = -2; %value for k
h_list = [2.5, 1, 0.75, 0.5, 0.3, 0.25]; % step sizes
t_final = 15; %Upper Bound of Evaluation

legend_list = strings(1, numel(h_list)); %Initialize the Legend Naming System
legend_index = 1;

%Visual Settings
colors = lines(length(h_list)); %Color for each k value

hold on; %Plot All on Same Graph

for h_index = 1:length(h_list) %Iterate though each h value

    h = h_list(h_index);
    color = colors(h_index,:); %Assign line style based on h

    %Create time range
    t = 5:h:t_final; % start at 5 instead of 0 to make xaxis log scale work

    %Prepare Internal Vars
    x = zeros(size(t));
    n = numel(t);

    x(1) = x_0; % set x(1) to initial x

    %Run Euler Iteration
    for i=1:n-1
```

---

```

        x(i+1) = x(i) + h * x_prime(t(i), x(i), k); %Evaluate the DE using
Euler's Method
    end

    %Apply symmetric log transform for +/- values
    x_plot = sign(x) .* log10(1 + abs(x)); % symlog style

    %Plot Approximations
    plot(t, x_plot, 'Color',color, 'LineWidth',1.5)

    %Append Legend Name
    legend_list(legend_index) = (" h = " + h);
    legend_index = legend_index + 1;

end
hold off;

%Graph Settings
ylabel('$x \rightarrow \mathrm{sign}(x) \cdot \log_{10}(1 + |x|)$', 'Interpreter', 'latex'); %LaTeX y-axis
xlabel('t'); %x-axis label
title('Euler Approximation (k = -2)') %Plot title
legend(legend_list, 'Location', 'eastoutside') %Add legend
grid on %Turn on grid
xscale log;

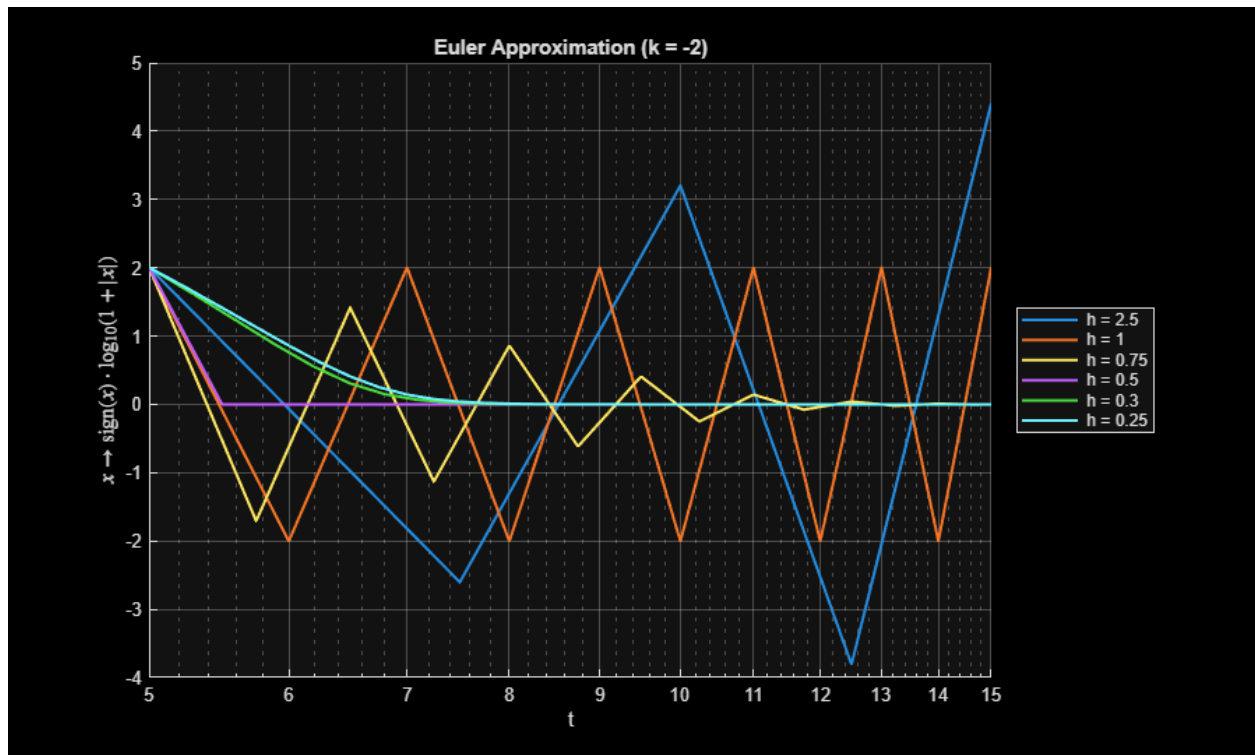
hold off;

% Response
% Q: Find the range of values for h such that the approximation from Euler's
%     method also captures this behaviour. What happens when h is outside of
%     this range?

% R: The value of h where this behavior is correctly represented is
%     h = (0, -1/(k)] = (0, 0.5]. There are different behaviors for
%     different values of h > -1/k. These Behaviours are as follows:
%     if h = (-1/k, (-2)/k) then the solver will oscillate with a
%     dampening effect. if h = (-2)/k, the solver will infinitely oscillate
%     between + and -. and if h > (-2)/k, the solver will oscillate with
%     blowout.

```

---



## Part 4

Response Q: Why might this cause problems when modelling a real-world situation? You might want to also briefly discuss the case where  $k$  is large and negative.

```
% R: When modeling real world situations, we do not have controll over
% the value of k. Because of this range of correct Euler behavior we
% found above, we can see how this would cause issues. If k is very
% large and negative, than the range where a solver wont ocillate is
%  $h = (0, -1/(k))$ . With a very large k, this means h must be very small
% leading to more computational intensity and more rounding error
% possibilities. Even if we only want the de to eventually reach 0, and
% dont care about ocilation, with a very large k, the range of
% acceptable values would be  $h = (0, (-2)/k)$  which is still very small
% given a large k value, meaning there is still significant
% computational intensity.
```

## Part 5:

R: Yes, this family does have a unique solution given a initial value. The work is on Rex's Remarkable. The fact that his equation has a clean unique solution  $[x(t) = x_0 \cdot e^{(kt)}]$  tells us that the errors in the Euler modeling in this equation stem from the method of modeling(euler) rather than a behavior of the equation itself.

*Published with MATLAB® R2025b*

## PART 6

---

```

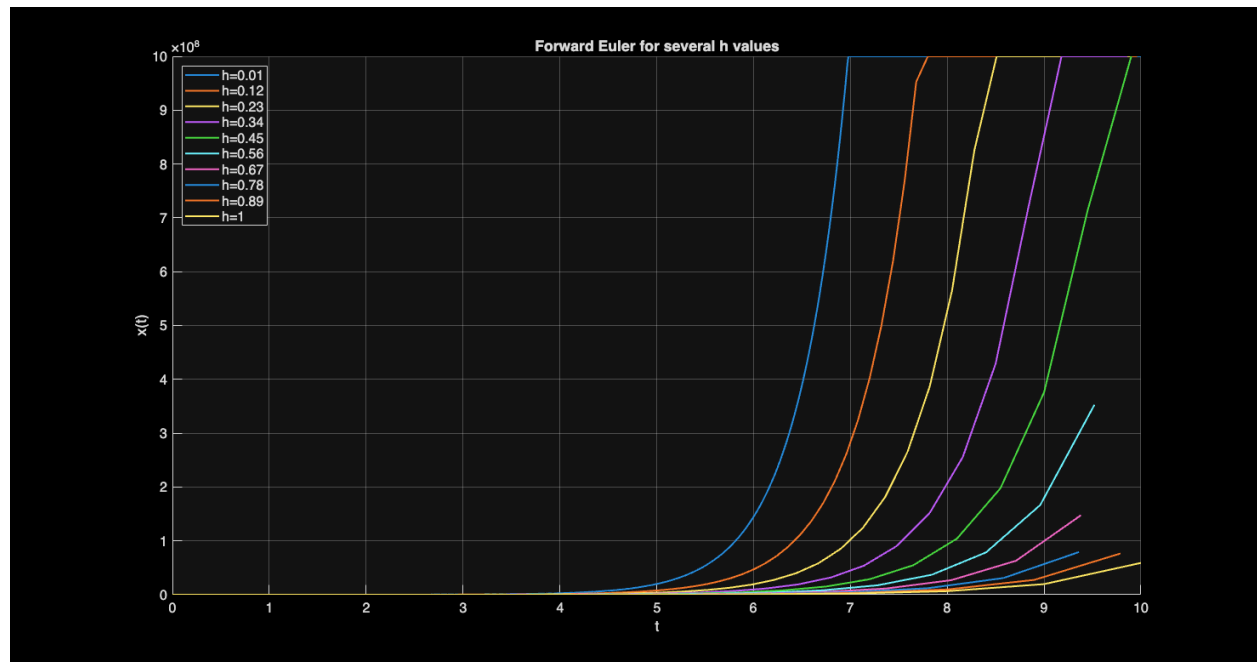
% iii) Use several values of h on the same plot
hs = linspace(0.01, 1, 10); % 10 different time steps between 0.01 and 1
k = 2;
x0 = 1000; % arbitrary positive x value
colors = lines(numel(hs));
ymax = 1e9; % Cap y to make all the lines visible

figure; hold on;
Tfinal = (numel(x)-1)*h; %Tfinal dependent on h
for j = 1:numel(hs)
    h_j = hs(j);
    t_j = 0:h_j:Tfinal;
    x_j = zeros(1, numel(t_j));
    x_j(1) = x0;
    for i = 1:(numel(t_j)-1)
        x_j(i+1) = x_j(i) + k * x_j(i) * h_j; %Euler step calculation
        if x_j(i+1) > ymax
            x_j(i+1) = ymax;
            % If capped, keep remaining values at cap and break integration
            if i+1 < numel(t_j)
                x_j(i+2:end) = ymax;
            end
            break;
        end
    end
    %Plotting
    plot(t_j, x_j, '-', 'Color', colors(j,:), 'DisplayName',
sprintf('h=%.3g', h_j), 'LineWidth', 1.5);
end

%Plot formatting
xlabel('t', 'FontSize', 14)
ylabel('x(t)', 'FontSize', 14)
title('Forward Euler for several h values', 'FontSize', 16)
legend('Location', 'northwest', 'FontSize', 12)
set(gca, 'FontSize', 12)
grid on
hold off

```

---



*Published with MATLAB® R2025a*

## **PART 7**



---

```

% DE Family:  $x' = kx(1-x)$ 

k = 2; % growth rate
h_vals = [1, 0.5, 0.1, 0.01]; % step sizes to test (updated to include h = 1)

lw = 1.5; % desired line width
fs = 14; % larger font size for axes and titles

figure;
for p = 1:numel(h_vals)
    h = h_vals(p);
    t = 0:h:10; % time vector
    % First initial condition
    x1 = zeros(1, numel(t));
    x1(1) = 0.5;
    for i = 1:(numel(t)-1)
        dx = k * x1(i) * (1 - x1(i));
        x1(i+1) = x1(i) + dx * h;
    end

    % Second initial condition
    x2 = zeros(1, numel(t));
    x2(1) = 2;
    for i = 1:(numel(t)-1)
        dx = k * x2(i) * (1 - x2(i));
        x2(i+1) = x2(i) + dx * h;
    end

    % Plot both solutions on the same subplot for this h
    subplot(numel(h_vals), 1, p)
    p1 = plot(t, x1, '-o', 'MarkerSize', 3, 'LineWidth', lw, 'DisplayName',
'x(1)=0.5'); hold on
    p2 = plot(t, x2, '-s', 'MarkerSize', 3, 'LineWidth', lw, 'DisplayName',
'x(1)=2');
    hold off
    xlabel('t', 'FontSize', fs)
    ylabel('x(t)', 'FontSize', fs)
    title(sprintf('Forward Euler, k = %d, h = %.3f', k, h), 'FontSize', fs)
    legend('Location', 'best', 'FontSize', fs-2)
    grid on
    set(gca, 'FontSize', fs)
end

% repeat for k = -2 to observe behavior with negative growth rate
k = -2;
figure;
for p = 1:numel(h_vals)
    h = h_vals(p);
    t = 0:h:10;
    x = zeros(1, numel(t));
    x(1) = 0.5; % initial condition

```

---

---

```

% Euler integration loop
for i = 1:(numel(t)-1)
    dx = k * x(i) * (1 - x(i));
    x(i+1) = x(i) + dx * h;
end

% Plot results for this h
subplot(numel(h_vals),1,p)
plot(t, x, '-o', 'MarkerSize', 3, 'LineWidth', lw)
xlabel('t', 'FontSize', fs)
ylabel('x(t)', 'FontSize', fs)
title(sprintf('Solution of dx/dt = kx(1-x) (forward Euler), k = %d, h =
%.3f', k, h), 'FontSize', fs)
grid on
set(gca, 'FontSize', fs)
end

% same negative k with different initial condition
k = -2;
figure;
for p = 1:numel(h_vals)
    h = h_vals(p);

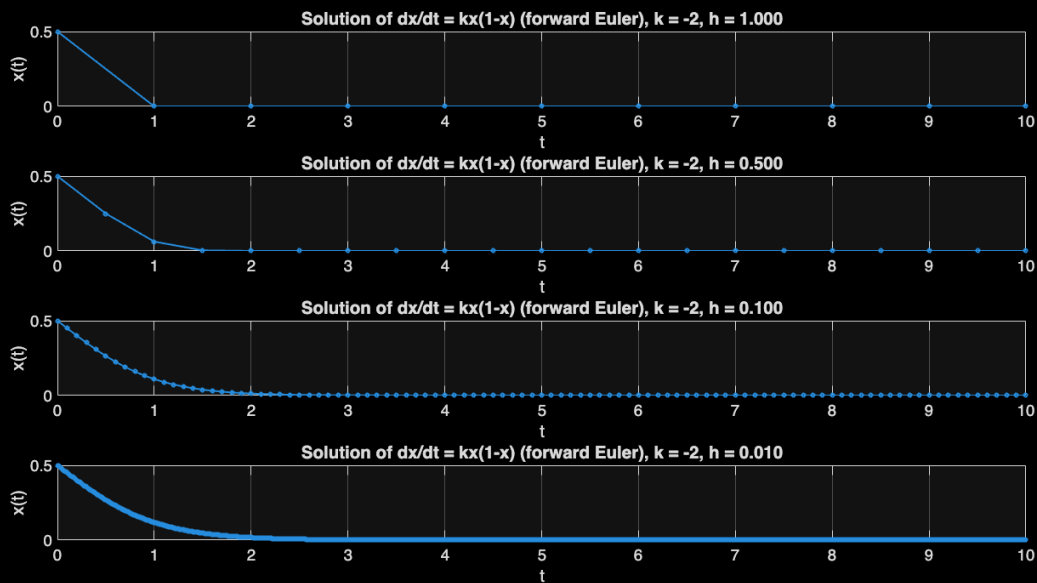
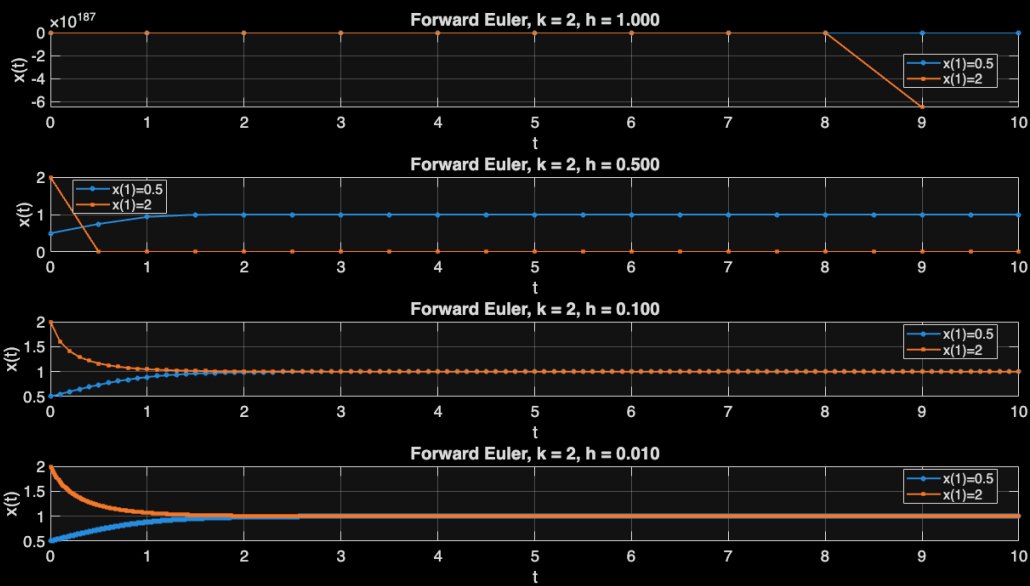
    Tfinal = h * 37;
    t = 0:h:Tfinal;
    x = zeros(1, numel(t));
    x(1) = 2; % initial condition > 1

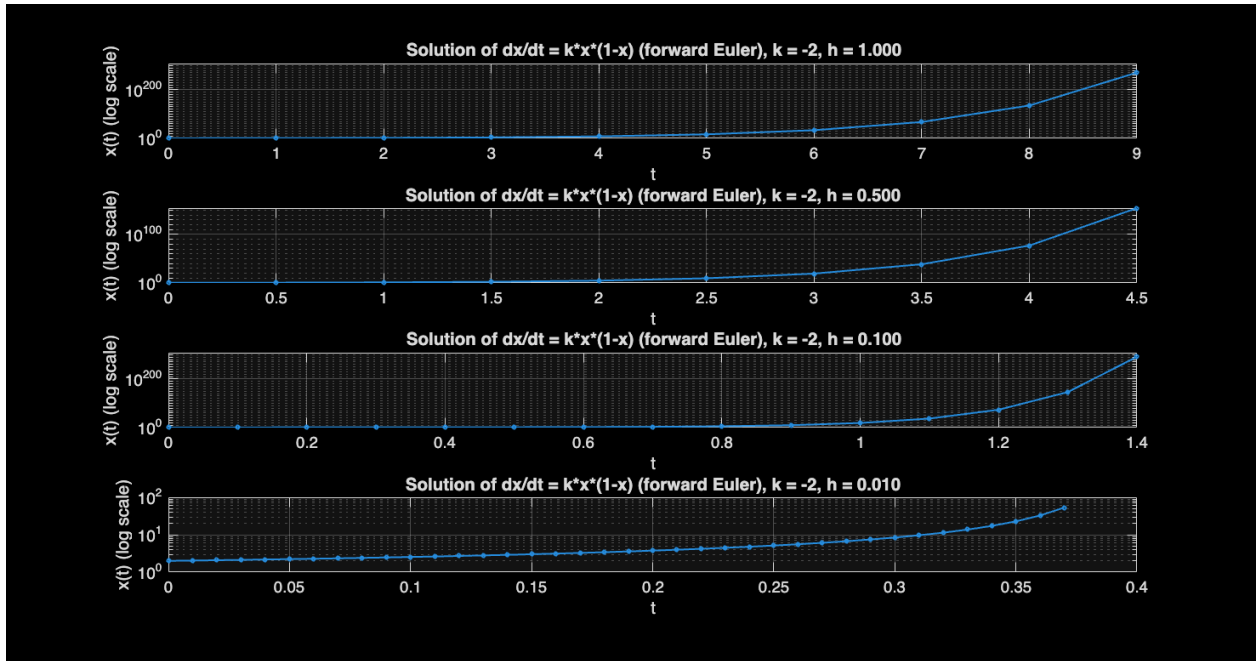
    % Forward Euler updates
    for i = 1:(numel(t)-1)
        dx = k * x(i) * (1 - x(i));
        x(i+1) = x(i) + dx * h;
    end

    % Plot each case (log scale for y-axis)
    subplot(numel(h_vals),1,p)
    semilogy(t, max(x, eps), '-o', 'MarkerSize', 3, 'LineWidth', lw) % avoid
log(0) by clamping to eps
    xlabel('t', 'FontSize', fs)
    ylabel('x(t) (log scale)', 'FontSize', fs)
    title(sprintf('Solution of dx/dt = k*x*(1-x) (forward Euler), k = %d, h
= %.3f', k, h), 'FontSize', fs)
    grid on
    set(gca, 'FontSize', fs)
end

```

---





*Published with MATLAB® R2025a*